



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Course – Software Engineering Lab (SE)

UID	2024800038
Name	Atharva Pramod Hule
Class and Batch	TY CSE Batch A3
Date	07/09/2026
Lab #	5
Aim	To analyze the requirements of a system by designing a Data Flow Diagram (level 0 and level 1)
Objective	1. Prepare a Level 0 DFD also known as Context Diagram 2. Identify the external entities, data input to the system by external entities and output data generated by the system 3. To factor or explode the level 0 DFD to identify the high level functions, input and output of each function and represent the same 4. Generate the data dictionary for the Data Flow diagram
Theory	A UML Activity Diagram is an UML diagram. It is used to model the workflow of a system or process. This type of diagram shows how a process starts moves through a series of activities makes decisions runs operations and finally reaches an outcome. Unlike a class diagram, which shows the structure of a system an activity diagram focuses on what happens during the execution of a process and how control flows from one activity to another. It can be used to model software workflows, business processes, algorithms use cases and operational procedures. The main building block of an activity diagram is an Action or Activity. This represents an unit of work or operation. It is usually shown as a rectangle. Activities are connected using Control Flows. These are represented by directed arrows that show the order in which activities happen. The workflow begins with an Initial Node. This is shown as a filled circle. It normally ends with an Activity Final Node. This is represented by a bull's-eye symbol. For example in a shopping system the activities may follow this path: Browse Products → Add to Cart → Make Payment → Order Confirmation. A Decision Node is shown as a diamond. It is used when the next activity depends on a condition. It has one flow and several possible outgoing flows. Each outgoing path can have a Guard Condition. This is written in brackets like [Payment Successful] or [Payment Failed]. It tells us when that path should be taken. A Merge Node is also shaped like a diamond. It serves a different purpose. It brings paths back into one flow. It does not synchronize anything. For instance in a payment process after checking the payment the



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

	system might take the [Payment Successful] path to order confirmation or the [Payment Failed] path to an error message. An activity diagram can also show concurrent activities. This is done using a Fork Node. A fork appears as a bar. It takes one flow and splits it into multiple outgoing flows. This allows those activities to run at the time. A Join Node, also shown as a bar brings these parallel flows back together. It waits for all the parallel activities to finish before letting the process continue. For example during airline check-in checking the passport and weighing the baggage can happen in parallel. The process only moves to boarding both are complete. So a fork stands for execution while a decision stands for choosing one path based on a condition. Swimlanes, also known as activity partitions help show who or which component is responsible for each activity. The diagram is divided into lanes for different users, roles, departments or system components. An activity placed inside a lane belongs to the participant shown in that lane. For instance an insurance claim workflow might have lanes for the Customer, Claims Handler and Accounting Department. This makes it easy to see who does what and how the workflow moves between participants. So an activity diagram gives a visual picture of a full workflow. It combines activities, control flow, decision and merge paths, guard conditions, parallel execution using fork and join and responsibility using swimlanes. This kind of diagram is especially helpful, for understanding system behavior. It helps find steps or inefficiencies. It also makes it easier to communicate the workflow among developers, users and other stakeholders.
Implementation / Code	<pre>@startuml title Green-Finance - Activity Diagram User / Swagger UI start :Open Swagger UI; :Send authentication request; Backend Services :Validate user credentials; if (Authentication successful?) then ([Yes]) User / Swagger UI :Display authentication response; :Send payment/transaction request; Backend Services </pre>



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

```
:Receive transaction request;
:Process simulated transaction;
:Increment transaction counter;

fork

    |Monitoring & Metrics|
    :Measure container energy using eBPF/RAPL;

fork again

    |Monitoring & Metrics|
    :Scrape application request metrics;

end fork

:Store transaction and energy metrics;

|Carbon Analysis & Dashboard|

:Retrieve transaction and energy metrics;
:Apply regional carbon factor;
:Calculate Carbon-per-Transaction;

if (Carbon-per-Transaction exceeds threshold?) then ([Yes])

    :Display Carbon-per-Transaction;
    :Display high-emission alert;

else ([No])

    :Display Carbon-per-Transaction;
    :Display normal status;

endif

|User / Swagger UI|
:Update/view dashboard;

stop

else ([No])

    |User / Swagger UI|
    :Display authentication failure;
```



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

stop

endif

@enduml

The swimlanes and corresponding activities in the above activity diagram are as follows:

Swimlane	Activities Performed	Description
User / Swagger UI	1. Open Swagger UI 2. Send authentication request 3. Display authentication response 4. Display authentication failure 5. Send payment/transaction request 6. Update/view dashboard	- Accesses the Swagger UI to interact with the system. - Sends login credentials for authentication. - Shows a successful authentication response to the user. - Displays an authentication failure message if credentials are invalid. - Initiates a payment/transaction request after successful login. - Views the final Carbon-per-Transaction result and dashboard.
Backend Services	1. Validate user credentials 2. Receive transaction request 3. Process simulated transaction 4. Increment transaction counter	- Verifies the user's credentials (Auth Service). - Accepts the transaction request from the user. - Processes the simulated financial transaction (Payment Service). - Increments the transaction counter (e.g., http_requests_total).
Monitoring & Metrics	1. Measure container energy using eBPF/RAPL 2. Scrape application request metrics 3. Store transaction and energy metrics	- Kepler measures the container's energy consumption using eBPF/RAPL. - Prometheus scrapes the application request metrics (e.g., request count). - Stores both transaction and energy-related metrics for further analysis.
Carbon Analysis & Dashboard	1. Retrieve transaction and energy metrics 2. Apply regional carbon factor 3. Calculate Carbon-per-Transaction 4. Check if Carbon-per-Transaction exceeds threshold 5. Display Carbon-per-Transaction (high-emission path) 6. Display high-emission alert 7. Display Carbon-per-Transaction (normal path) 8. Display normal status	- Retrieves the required metrics from Prometheus. - Applies the appropriate regional carbon factor. - Calculates the Carbon-per-Transaction using PromQL. - Evaluates whether the calculated value exceeds the configured threshold. - Displays the Carbon-per-Transaction value when the threshold is exceeded. - Raises a high-emission alert in case of high carbon intensity. - Displays the Carbon-per-Transaction value when the threshold is not exceeded. - Shows a normal status indicating acceptable carbon intensity.



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Output	Green-Finance - Activity Diagram
Conclusion	The UML Activity Diagram was successfully designed to represent the workflow of the Green-Finance system. The diagram clearly shows the sequence of activities starting from user authentication moving through transaction processing to energy monitoring, metric collection and finally to Carbon-per-Transaction calculation and visualization. Decision nodes are used to show conditions like authentication success or whether emission thresholds are met. Fork and join nodes illustrate operations such as measuring energy and collecting metrics at the same time. Swimlanes help clarify who is responsible for each task—whether it's the user, a service, a monitoring tool or the visualization component. This layout makes it easy to see how different parts of the system work together. As a result the activity diagram gives an organized view of the system's dynamic behavior. It helps explain the flow of actions the choices that can be made the tasks that happen at the time and the end results, in a way that is easy to follow.
References	https://dev.to/edgaras/uml-activity-diagrams-4cmj https://miro.com/diagramming/what-is-a-uml-activity-diagram/ https://www.getharvest.com/blog/activity-diagram